数学与 CS 问答笔记

answer-note

按日归档的问答笔记，承接 2026-10-16：今天用 software-construction 的“写程序”配方（6.102 R07 · 6.031 R19）把 KV 的构建流程走一遍——只写规格与测试设计，不代写实现。

October 17, 2026

Contents

1. 当日起点：项目状态与工具状态 3
2. 用 Go 走一遍构建流程：规格 → 测试 → 实现（Writing a Program with ADTs） | 6.102 R07 · 6.031 R19 4
   - 2.1. 费曼直觉 4
   - 2.2. 硬核机制 4
     - 2.2.1. 材料的三层配方 4
       - 2.2.1.1. 第一层：写一个方法（Writing a method） 4
       - 2.2.1.2. 第二层：写一个 ADT（Writing an abstract data type） 5
       - 2.2.1.3. 第三层：写一个程序（Writing a program） 5
       - 2.2.1.4. 材料用的例子：矩阵表达式 5
       - 2.2.1.5. 材料 Summary 的三支柱 6
     - 2.2.2. 套到 KV：四张你要亲手填的表 6
       - 2.2.2.1. 表 1：操作分类 6
       - 2.2.2.2. 表 2：规格契约（空位由你填） 6
       - 2.2.2.3. 表 3：AF / RI / 表示暴露 7
       - 2.2.2.4. 表 4：测试分区（输入空间划分） 7
   - 2.3. 落到项目：今天只产出“规格 + 测试设计” 8
     - 2.3.1. 现场事实：红在哪一层 8
     - 2.3.2. 三个必须你拍板的规格决策 9
     - 2.3.3. 你的执行顺序（五步，每步只做一件事） 9
     - 2.3.4. 验收清单（Definition of Done） 10
   - 2.4. 陷阱与易错 10
   - 2.5. 关键点 11
   - 2.6. 变式检验 11
   - 2.7. 同类归类 11
   - 2.8. Go 语言机制补充 12
     - 2.8.1. 多返回值与 `error` 12
     - 2.8.2. `delete` 的语义 12
     - 2.8.3. map 索引与 `string(key)` 的拷贝 12
     - 2.8.4. `nil` map 与零值 12
     - 2.8.5. 工具链的三条边界 12
   - 2.9. 本次操作与当前状态 12
3. 交互式练习：KV 构建的第一批题目 14
   - 3.1. 本轮实际改动 14
   - 3.2. 交作业与检验方式 14

Index of Listings 15

1. 当日起点：项目状态与工具状态

10-16 收尾时留下的是两条待办：补 `Del` 的函数体、把 `Get` 未命中的返回值改成测试要求的形状。今天不急着做这两步——先把“为什么要这样做”补上。现场复核如下：

项目项	当前状态	今天实测
`internal/db/kv.go`	`Del` 只有签名（第 29 行），包编译不过	`go build ./...`：`missing function body`
`internal/db/kv_test.go`	第 22、31 行要求“未命中不算错误”	与实现的 `errors.New` 冲突
`go vet ./internal/db/`	通过	**静默**，不报缺函数体
`gofmt -l internal/db/`	无输出	格式已合格
10-15 的两条待办	① 补 `Del`；② 改 `Get` 未命中语义	均未做（今天也不做）

> 📌 **核心要点** 今天的问题不是“再补一个方法”，而是**把构建流程本身走一遍**：规格（spec）→ 测试（test）→ 实现（implement）。你手上恰好有一个“半测试、半实现”的现场，正好当教材。

> ⚠️ **陷阱预警** 按你的要求，**本轮不改任何实现代码**。下面所有代码块都是“模板/待填”，不是答案；真正的函数体由你自己写。

2. 用 Go 走一遍构建流程：规格 → 测试 → 实现（Writing a Program with ADTs）｜6.102 R07 · 6.031 R19

2.1. 费曼直觉

把“写代码”想成装修：

1. **规格（Specifications）** 是设计图：门在哪、墙多高、哪些是承重墙；
2. **测试（Testing）** 是验收单：开门不响、墙面垂直、插座有电；
3. **实现（Implementation）** 是施工。

没有设计图，验收单和施工队就各凭想象。你现在的 `KV` 正是这样：施工队（`Get`）认为“找不到钥匙”是事故，要拉警报（返回 `error`）；验收单（`kv_test.go`）却认为“找不到钥匙”是正常情况（`ok == false`、`err == nil`）。两边都没错——错在**没有一张双方都认的图**。

所以构建流程的第一动作不是打开编辑器写函数体，而是把“应该怎样”写下来。一句话：

> 📌 **核心要点 构建 = 先定义“应该”，再采样“应该”，最后兑现“应该”；规格、测试、实现三者互相施压，直到重合。**

> 💡 **背景与类比** 为什么测试要“先红”？因为一个从未失败过的测试，不能证明它能发现 bug——它可能只是恰好和错误实现一样错。材料原话：**Make sure at least some of your tests are failing at first.**

2.2. 硬核机制

2.2.1. 材料的三层配方

6.031 R19 把“写程序”拆成三层递进的配方（recipe）：写一个方法、写一个 ADT、写一个程序。三层不是三选一，而是**同一个循环在不同尺度上的重复**。

2.2.1.1. 第一层：写一个方法（Writing a method）

材料原文的三步：

```
Spec. Write the spec, including the method signature (name, argument types, return types, exceptions), and the precondition and the postcondition as a Javadoc comment.

Test. Create systematic test cases and put them in a JUnit class so you can run them automatically.

Implement. Write the body of the method. You're done when the tests are all green in JUnit.
```

翻译到 Go：

步	材料（Java/JUnit）	Go 里怎么做
Spec	Javadoc 注释写前置/后置	`//` doc comment；`go doc` 能读出来
Test	JUnit 测试类	`_test.go` + `testing` + `testify/assert`

Implement	方法体；JUnit 全绿	函数体；`go test` 输出 `ok`

材料紧接着补了一句关键的话：三步会**来回弹**——写测试会施压规格，写实现又会施压测试和规格。这不是流程失败，正是流程在工作。这就是测试优先（test-first programming）。

> ⚠️ **陷阱预警** 材料还要求：**测试套件在实现之前必须至少有一个是红的**。如果你的测试在函数体还是空的时候全绿，那它测的不是行为，而是“什么都没发生”。

2.2.1.2. 第二层：写一个 ADT（Writing an abstract data type）

ADT 层的配方把“实现”这一步展开：

1. **Spec**：给每个操作写签名、前置条件（precondition）、后置条件（postcondition）；
2. **Test**：给操作写测试；测试会逼你发现“还缺哪个操作”；
3. **Implement**：
   1. 选表示（representation，rep）：私有字段/递归变体；
   2. 写下表示不变式（rep invariant, RI）和抽象函数（abstraction function, AF）；
   3. 实现 `checkRep` 强制执行 RI；
   4. 实现每个操作，并在操作里调用 `checkRep`；全绿才算完。

Go 的映射：ADT = 非导出字段的结构体 + 导出方法；RI 检查写成方法；RI 被破坏时 `panic`（fail fast），因为那是**程序员 bug**，不是调用方错误。

2.2.1.3. 第三层：写一个程序（Writing a program）

程序的配方是前两层的放大：

步	要做的决策
1. 选数据类型	哪些可变、哪些不可变；先定边界
2. 选静态方法	从 `main` 往下拆步骤，一步一个函数
3. 写规格	ADT 和方法的规格；操作先少而简单，需要时再加
4. 写测试	每个单元（ADT 或方法）都要有
5. 先简单实现	暴力表示优先；先让整个程序跑通，跳过优化与边角，记 TODO
6. 迭代	跑通后再重构、优化、重新设计

> 📌 **核心要点** 第三层的灵魂是 **simple first**：先用最笨的表示把全链路跑通，再谈优化。你现在的 `map[string][]byte` 就是最笨、也最合适的表示——不要在这上面提前加锁、加缓存、加持久化。

2.2.1.4. 材料用的例子：矩阵表达式

材料用矩阵表达式（MatExpr）演示整套配方：`make`（标量或矩阵）、`times`、`isIdentity`、`optimize`。每个操作先写签名 + requires + effects；例如 `optimize` 的后置条件是“返回一个值相同、但可能算得更快的表达式”。注意它**没有规定怎么优化**——规格只谈行为，不

谈实现。这正是你写 `Set` / `Get` 规格时要学的：先定“调用方能看到什么”，再谈内部怎么写。材料的 More practice 是矩阵乘法的额外练习；今天用 KV 的分区表（表 4）替代。

2.2.1.5. 材料 Summary 的三支柱

材料每讲用三条价值收尾，正好对应 KV 的三层缺口：

1. **Safe from bugs（免于 bug）**：规格 + 分区测试 + RI 检查，把 bug 挡在交付前；
2. **Easy to understand（易于理解）**：规格就是文档，测试策略就是可读的用例命名；
3. **Ready for change（易于修改）**：表示独立，换 rep 不换契约。

2.2.2. 套到 KV：四张你要亲手填的表

2.2.2.1. 表 1：操作分类

抽象数据类型（abstract data type, ADT）由它的操作定义。四分类是：创建器（creator）、变值器（mutator）、观察器（observer）、生产者（producer）。

操作	分类	说明
`Open`	创建器	建立一个可用的空映射
`Set`	变值器	插入或覆盖一个键值对
`Del`	变值器	删除一个键
`Get`	观察器	查询，不改变抽象值
`Close`	生命周期	释放资源；可重复调用吗？由你定

注意 `Close` 不在四分类里——它是资源管理，不是抽象值上的操作。写规格时要说清“Close 之后还能不能调用其他方法”，否则调用方只能猜。

2.2.2.2. 表 2：规格契约（空位由你填）

规格 = 前置条件 + 后置条件 + 失败模式。Go 没有 checked exception，可预期的失败用返回值表达，所以“返回什么”本身就是规格的一部分。

方法	REQUIRES（前置）	EFFECTS（后置）	失败模式
`Open`	待填	待填	待填
`Get`	待填	待填	待填
`Set`	待填	待填	待填
`Del`	待填	待填	待填
`Close`	待填	待填	待填

填表时每个方法至少要回答这些问题：

1. `Open`：重复调用是清空还是保留？可能失败吗？
2. `Get`：未命中返回什么？`nil` key、空 key 合法吗？
3. `Set`：`updated` 的定义是什么？`nil` 值合法吗？
4. `Del`：`deleted` 的定义是什么？不存在时返回什么？
5. `Close`：重复调用安全吗？之后还能调用其他方法吗？

⚠️ **陷阱预警** 这张表不是“抄一遍现有代码”。现有代码只覆盖了部分行为；未定义的地方（重复 `Open`、覆盖写、空 key、`nil` 值、`Close` 后调用）必须由你**做决定**，再写进注释。规格是决策，不是描述。

2.2.2.3. 表 3：AF / RI / 表示暴露

表示（rep）是字段的具体值；抽象函数（AF）说明“这组字段值代表哪个抽象值”；表示不变式（RI）说明“哪些字段值组合是合法的”；表示暴露（rep exposure）说明“客户端能不能绕过抽象边界改到 rep”。

项	当前现场	你要写下的
rep	`mem map[string][]byte`	私有字段；key 的字节从哪来
RI	没有任何声明	`Open` 后 `mem != nil`；是否允许 `nil` 值？
AF	没有	`AF(mem)` = 每个 key 映射到哪串字节
表示暴露	`Get` 返回内部切片；`Set` 保存外部切片	拷贝，或明确声明“借用”
`checkRep`	不存在	RI 被破坏时 `panic`

```
// KV is a mutable in-memory key-value store.
//
// Abstraction function:
//   AF(mem) = 待填
// Rep invariant:
//   mem ≠ nil after Open.
// Safety from rep exposure:
//   待填

type KV struct {
  mem map[string][]byte
}
```

Listing 1: ADT 文档模板：先填注释，再谈函数体

2.2.2.4. 表 4：测试分区（输入空间划分）

测试不是“随便想几个例子”，而是把输入空间切成若干**分区**（partition），每个分区再切**子域**（subdomain），每个子域至少一个用例。边界（boundary）单独列。黑盒（black-box）测试只看规格选用例；白盒（glass-box）测试再看实现补边界。

分区	子域	当前测试覆盖	建议用例名
P1 生命周期	未 Open / Open 后 / Close 后	只覆盖 Open 后	before open / after close
P2 key	nil / 空 / 非空	只覆盖非空	empty key / nil key

P3 值	nil / 空切片 / 非空	只覆盖非空	nil value / empty value
P4 `Set`	新键 / 覆盖不同值 / 覆盖同值	只覆盖新键	overwrite changed / overwrite same
P5 `Get`	命中 / 未命中 / 删除后	命中、未命中	get missing / get deleted
P6 `Del`	存在 / 不存在	不存在、存在（待实现）	del missing / del present
P7 别名	调用后改 key/val	无	mutate after set
P8 重复 `Open`	第二次 `Open`	无	reopen

```go
// Testing strategy for KV:
//   partition on operation: Open, Set, Get, Del, Close
//   partition on key: absent, present, empty, nil
//   partition on value: nil, empty, non-empty
//   partition on Set: new key, overwrite different value, overwrite same value
//   partition on Del: present, absent
//   partition on aliasing: caller mutates key or value after the call
//   partition on lifecycle: before Open, after Open, after Close
//
// Each test case should name the subdomains it covers.
func TestKVBasic(t *testing.T) {
  // TODO: 按上表补用例
}
```

Listing 2: 测试策略注释模板：把“为什么选这些用例”写在测试文件顶部

📌 **核心要点** 分区的价值在于**暴露缺口**。当前 `kv_test.go` 的 6 个断言只覆盖了 3 个分区（生命周期、存在性、删除存在性）；上表右边两列几乎全空——这就是“测试绿了也不安心”的根源。

2.3. 落到项目：今天只产出“规格 + 测试设计”

2.3.1. 现场事实：红在哪一层

位置	代码事实	后果	属于哪一层
`kv.go:29`	`Del` 只有签名	编译失败	实现层
`kv.go:16-22`	`Get` 未命中返回 `error`	与 `kv_test.go:22`、`:31` 冲突	规格层
`kv.go:24-27`	`Set` 恒返回 `true`	覆盖写语义未定义	规格层
`kv.go:5-7`	`mem` 没有任何 RI/AF 注释	别名与生命周期靠猜	ADT 层
`kv_test.go`	6 个断言点	测试只采样 3 个分区	测试层

📌 **核心要点** 结论：**缺的不是一行函数体，而是三层各缺一块**——实现层缺 `Del`，规格层缺未命中/覆盖语义，ADT 层缺 RI/AF/别名约定。补 `Del` 只能让编译器闭嘴，补不了后两层。

2.3.2. 三个必须你拍板的规格决策

1. **`Get` 未命中：返回 `error` 还是 `(nil, false, nil)`？** 测试已经投票给后者（`kv_test.go:22`、`:31`）。推导：Go 的 `error` 表示“发生了意外失败”（I/O、损坏、权限），而“这个 key 不存在”是映射的正常分支；把正常分支塞进 `error`，调用方每次都得先判 `err`，容易把“没有”当成“坏了”。**决策：未命中 = `(nil, false, nil)`；`error` 保留给真正的故障。**
2. **`Set` 的 `updated`：表示“写入成功”还是“值发生变化”？** 两种定义都能让当前测试过（它只测新增键）。区别在覆盖写：定义 A 恒 `true`；定义 B 在“新键或值不同”时 `true`，覆盖同值时 `false`。后者信息量更大，也更符合课程语义；代价是要读旧值比较（`bytes.Equal`）。**这是规格决策，先写下来再实现。**
3. **所有权：`Set` / `Get` 是“借用”还是“拥有”切片？** 当前是借用：`Set` 直接把 `val` 存进 map，`Get` 直接返回内部切片。调用方改一下 `val[0]`，存储内容就跟着变——这是表示暴露。两条路：A 明确文档“借用，调用方不得修改”；B 在 `Set` / `Get` 里拷贝（`bytes.Clone` / `slices.Clone`）。简单优先：当前阶段选 A + 写清文档；等做并发或持久化时再升级到 B。

⚠️ **陷阱预警** 三个决策都必须落到 `kv.go` 的注释里，并让测试覆盖到。只写在聊天里不算数—— `go doc ./internal/db KV` 读不到的决定，等于没做。

2.3.3. 你的执行顺序（五步，每步只做一件事）

步	动作	产物	验证命令
1	写 `KV` 的 AF/RI/表示暴露注释	ADT 文档	`go doc ./internal/db KV`
2	给五个方法写 requires/effects	规格契约	`go doc ./internal/db`
3	按表 4 补测试用例	先红的测试	`go test -count=1 ./internal/db/`
4	只补 `Del` 函数体	编译通过	`go build ./...`
5	依次修 `Get` 未命中、`Set` 的 `updated`	测试全绿	`go test -count=1 ./internal/db/`

📌 **核心要点** 顺序不能颠倒：**先规格，再测试，最后实现**。如果你先补 `Del`，你会不自觉地按现有实现的思路写，规格就被实现反向绑架了。

每步的“停手条件”：

1. 第 1 步：`go doc` 能读到你写的 AF/RI/rep exposure；

2. 第 2 步：每个方法都有 requires/effects，未定义的地方写“未定义”而不是留空；
3. 第 3 步：新用例在实现未改时至少有一个红；
4. 第 4 步：`go build ./...` 退出码 0；
5. 第 5 步：`go test -count=1 ./internal/db/` 输出 `ok`，且 `go test -race ./internal/db/` 也 `ok`。

2.3.4. 验收清单（Definition of Done）

```
go build ./...
go test -count=1 ./internal/db/
go test -race ./internal/db/
go vet ./...
gofmt -l internal/db/
go test -cover ./internal/db/
```

Listing 3: 交付前的六条命令：前三条是硬门槛，后三条是质量信息

> ⚠️ **陷阱预警** `go vet` 通过**不等于**能编译：今天实测 `go vet ./internal/db/` 退出码 0，而 `go build ./...` 报 `missing function body`。`go vet` 做静态分析，不替代编译器；`go test` 才会真正编译测试包。

修绿之后，把这 6 个断言和后来补的用例全部留下——它们就是回归测试（regression testing）：以后改实现时，先让它们再跑一遍。

2.4. 陷阱与易错

1. **把“编译通过”当“构建完成”**：编译只是第一道门；规格、测试、RI 都还没验。
2. **把“测试绿”当“规格正确”**：测试是采样，不是证明。当前 6 个断言覆盖不到覆盖写、空 key、别名、重复 `Open`。
3. **用 `error` 表达“未命中”**：正常控制流被当成异常，调用方容易漏判；Go 的 `error` 是值，不是异常。
4. **`delete` 不告诉你删没删到**：`delete` 对不存在的键是安全空操作，所以 `deleted` 必须先查一次（`go doc builtin.delete` 原文：**If m is nil or there is no such element, delete is a no-op.**）。
5. **`nil` map 可读不可写**：`KV{}` 不 `Open` 就 `Set` 会 panic；`Get` 反而安全。防御要加在写的一侧。
6. **`[]byte` 不能直接做 map key**：`[]byte` 不可比较，必须转 `string`；转换会拷贝 key 的字节，所以 key 安全，但 **value 仍然是别名**。
7. **`updated` 恒 `true` 是“实现替规格做决定”**：测试碰巧过，语义却没定义；以后加缓存/持久化时会反咬。
8. **一次写五个方法**：违反 simple first；红了不知道是哪个变化引入的。一步一绿。
9. **过早加锁/优化**：当前是单线程内存 KV，先跑通；并发是 T16/T18 的话题。
10. **规格只写在脑子里**：未来的你、面试官、合作者都读不到；`go doc` 读不到的规格等于不存在。

2.5. 关键点

> 📌 **核心要点 一句话灵魂**：构建流程 = 规格定义“应该”，测试采样“应该”，实现兑现“应该”，三者互相施压直到重合。

1. 方法层：Spec → Test → Implement；测试必须至少先红一次。
2. ADT 层：先选 rep、写 RI/AF、实现 `checkRep`，再写操作。
3. 程序层：先简单跑通，再迭代优化；边角与性能记 TODO。
4. 你当前的瓶颈不是 `Del` 的函数体，而是 `Get` 的规格冲突和 `Set` 的 `updated` 未定义。

2.6. 变式检验

1. **材料 R19 的分区**：矩阵表达式 `optimize` 按“scalar 数量 0/1/2/>2”和“scalar 在树中的位置”分区。类比到 `KV`：如果按“操作序列长度 0/1/2/>2”分区，你会补哪些用例？（提示：单次 `Set` / `Get` 只是长度 1；覆盖写、删除后重插、重复 `Open` 都在长度 ≥2。）
2. **`Get` 的两种调用风格**：若未命中返回 `error`，调用方要写“先判 `err` 再判 `ok`”；若返回 `(nil,false,nil)`，只需判 `ok`。哪种更容易漏判？为什么 Go 的 comma-ok 惯用法流行？
3. **覆盖缺口**：对照表 4，列出当前 `kv_test.go` 至少 5 个未覆盖子域，并为每个写一条用例名（不用写函数体）。
4. **别名实验**：如果 `Set` 保存 `val` 的别名，调用者执行 `val[0] = 'x'` 后，`Get` 会看到什么？这违反哪条 RI 或规格？把答案写成一句话注释。
5. **`deleted` 的两种定义**：定义为“键存在”还是“删除动作发生”？在单线程下两者有区别吗？在并发下呢？（提示：查到存在之后、删除之前，别人可能先删。）

2.7. 同类归类

今天用的配方	同源的讲	项目里的实例
Spec / 前置后置	T05 规格（6.102 R04 / 6.031 R06）	`Get` 未命中契约
分区测试	T02 测试（6.102 R02 / 6.031 R03）	`kv_test.go` 的 6 个断言
操作四分类	T08 ADT（6.102 R06 / 6.031 R10）	`Open/Set/Get/Del`
AF / RI / 表示暴露	T09（6.102 R07 / 6.031 R11）	`mem` 的 RI 与切片别名
可变性与别名	T07（6.031 R08）	`Set` 保存 `[]byte`
综合演练	T23（6.102 R07 / 6.031 R19）	今天的整条流程

把这张表倒过来看：**后续的 `sorted_file.go`、`table.go`、`sql_parser.go` 都会重复同一套配方**——先规格、再分区测试、再简单实现、再迭代。今天在 `KV` 上把流程走顺，后面只是换素材，不是重新发明方法。

2.8. Go 语言机制补充

2.8.1. 多返回值与 `error`

`go doc builtin.error` 的原文：`error` 是约定接口，**the nil value representing no error**。Go 没有异常；可预期的失败用返回值表达，`panic` 只用于“程序自己坏了”（RI 被破坏、不可能分支）。所以 `Get` 的未命中到底算“失败”还是“正常结果”，是一个**规格决策**，不是语法问题。

2.8.2. `delete` 的语义

`go doc builtin.delete` 的原文：**The delete built-in function deletes the element with the specified key (m[key]) from the map. If m is nil or there is no such element, delete is a no-op.** 由此可推出：`delete` 不返回“是否删到”，`deleted` 必须在调用前用 comma-ok 查。声明在 `/usr/lib/go/src/builtin/builtin.go`。

2.8.3. map 索引与 `string(key)` 的拷贝

1. `v := m[k]` 单值形式：不存在时得到零值，分不清“没有”和“存了零值”；
2. `v, ok := m[k]` 双值形式（comma-ok）：`ok` 直接回答存在性——这是 `Get` 应该用的形式；
3. `[]byte` 不可比较，不能做 map key；`string(key)` 把字节复制成字符串，所以 **key 的别名风险被转换消除了**；
4. 但 value 是 `[]byte` 切片，直接存入 map 仍与调用方共享底层数组——这是 T07 讲的别名（aliasing），要靠拷贝或文档声明解决。

2.8.4. `nil` map 与零值

map 的零值是 `nil`。读 `nil` map 合法（返回零值），写 `nil` map 会 panic。`Open` 的职责就是把 `mem` 变成非 nil；这也是一条天然的 RI。`KV{}` 不 `Open` 就 `Set` 的实测见 10-13 笔记。

2.8.5. 工具链的三条边界

1. `go build` 是编译的权威：缺函数体、类型错误都在这里暴露；
2. `go vet` 做静态分析，**不替代编译**：`go vet ./internal/db/` 今天实测静默通过，但 `go vet ./...` 会报同一个 `missing function body`——检查范围取决于包加载方式，别把单包通过当全模块通过；
3. `go test` 会编译测试包并运行；`-race` 开竞态检测（本机 `CGO_ENABLED=1`），`-cover` 给覆盖率；`gofmt -l` 只列出未格式化的文件。

权威出处：`go doc builtin.delete`、`go doc builtin.error`、`go doc bytes.Clone`、`go help testflag`、`/usr/lib/go/src/builtin/builtin.go`。

2.9. 本次操作与当前状态

1. **新建** `note/2026-10-17.typ`，按 answer-note 完整头部；编译出同名 `.pdf`。
2. **未改动** `internal/db/kv.go`、`internal/db/kv_test.go`——遵守“不要替我实现”。
3. **实测**：

1. `go vet ./internal/db/` → 通过（退出码 0，且不报 `missing function body`）；但 `go vet ./...` → 报同一个 `missing function body`（检查范围不同）；
2. `gofmt -l internal/db/` → 无输出；
3. `go build ./...` → `internal/db/kv.go:29:6: missing function body`（沿用 10-16 的已知红）。

4. **产出**：操作分类表、规格契约表（5 个方法 × requires/effects/失败模式）、AF/RI/表示暴露表、8 个分区的测试表、五步执行顺序、六条验收命令。
5. **待你动手**：按“五步”先写规格注释，再补测试，最后按红→绿实现；每步只跑一条验证命令。三个待拍板决策：`Get` 未命中、`Set` 的 `updated`、切片所有权。
6. **下一步**：修绿后把 `scratch/kvprobe` 跑起来，并把这轮的决定记进 10-16 说的“决策日志”。

3. 交互式练习：KV 构建的第一批题目

承接上一节的"规格 → 测试 → 实现"配方，把第一批练习写进代码。原则：**出题人的 `kv_test.go` 断言不能改**；我们只加注释、脚手架和后续的新测试。

练习	位置	你要做的事
0	`kv.go` 顶部 + `kv_test.go` 顶部	补 AF/RI/表示暴露、测试分区；只写注释
1	`kv.go` 的 `Del`	把 `panic` 换成实现，让两条 Del 断言变绿
2	`kv.go` 的 `Get` 未命中分支	按两条未命中断言改返回

> 📌 **核心要点** 本轮只加这三道；`Set` 的 `updated` 语义与新测试留到下一批。**每批只引入一个需要思考的变化**，填完保存后再交给我检验。

> ⚠️ **陷阱预警** `go test` 现在是红的，而且有两层：`kv_test.go:29` 的 Get 未命中失败，以及 `kv.go:57` 的 `Del` panic。**不要为了让测试闭嘴去改断言**；先做练习 1，再做练习 2。

3.1. 本轮实际改动

1. `internal/db/kv.go`：加练习 0 的 AF/RI/表示暴露模板、练习 2 的 Get 提示、练习 1 的 `Del` 脚手架（`panic` 占位）。
2. `internal/db/kv_test.go`：加测试策略模板与下一批练习预告；**没有改任何既有断言**。
3. 实测：`go build ./...` 通过；`go vet ./internal/db/` 通过；`gofmt -l internal/db/` 无输出；`go test -count=1 ./internal/db/` 在 `kv_test.go:29` 与 `kv.go:57` 两处红（预期）。

3.2. 交作业与检验方式

填好并保存后告诉我，我会：

1. 先跑 `go test -count=1 ./internal/db/`，看红在哪一层；
2. 再读你的注释和实现，只指出**问题**（契约不一致、边界漏了、别名没处理、测试没覆盖），不直接给答案；
3. 如果卡住，我给下一步的提示或更小的实验，仍然不贴函数体。

下一批预告（等这三道变绿）：`Set` 的 `updated` 语义、覆盖写测试、空 key / `nil` 值、删除后重插。

Index of Listings

Listing 1 ADT 文档模板：先填注释，再谈函数体 7
Listing 2 测试策略注释模板：把“为什么选这些用例”写在测试文件顶部 8
Listing 3 交付前的六条命令：前三条是硬门槛，后三条是质量信息 10